BUILD / UNDERSTAND / EXPLAIN

从 Python 研究脚本到 Agent 服务

LangGraph、LangChain、FastAPI 与 MCP

基于你的 A 股隔夜反转项目，逐层拆解真实代码。

---

01 **确定性研究核心**
数据校验 / 因子 / T+1 回测

02 **工具与流程**
输入契约 / LangGraph / 模型工具循环

03 **服务与协议**
FastAPI / 后台实验 / MCP / 浏览器

30 节 / 代码拆解 / 动手练习 / 面试追问

适合：会基础 Python，刚开始学习 Agent 的实习求职者。

2026.10.09 | PROJECT EDITION 0.2

依据本地源码制作；能力与限制均按实际实现说明。

阅读导航

按编号学习；PDF 侧栏书签可直接跳到章节。

01 阅读地图：先跑通，再解释，再重写 3
02 四个名字分别负责什么 4
03 整体架构：三个入口，一个计算核心 5
04 源码地图：应该按什么顺序打开文件 6
05 第一步：安装依赖并跑通离线流程 7
06 第二步：先理解因子和回测核心 8
07 第三步：先给工具定义输入契约 9
08 第四步：把计算封装成共享工具 10
09 第五步：LangGraph 的 state 和 node 11
10 第六步：把节点连成可执行的图 12
11 第七步：给图增加条件分支和循环 13
12 第八步：模型请求里到底发送了什么 14
13 第九步：看懂一次 tool_call 返回值 15
14 第十步：错误处理和事实边界 16
15 可选练习：用 LangChain 包装同一个工具 17
16 第十一步：FastAPI 把 Python 函数变成 HTTP 接口 18
17 第十二步：长任务先返回 202 和实验 ID 19
18 第十三步：线程池、队列容量和生命周期 20
19 第十四步：保存实验，而不仅是保存一段文字 21
20 第十五步：前端如何提交实验 22
21 第十六步：前端轮询、展示和错误恢复 23
22 第十七步：MCP 不是另一种 FastAPI 24
23 第十八步：从客户端真正调用 MCP 25
24 一次点击的完整路径：把各层串起来 26
25 验证：怎样证明自己写的东西能用 27
26 练习：新增一个你自己的波动率工具 28
27 五天学习计划：每天有一个可检查的产物 29
28 面试表达：两分钟讲清项目 30
29 面试追问：先理解，再回答 31
30 排障与继续阅读 32

LESSON 01 / BUILD IT STEP BY STEP

阅读地图：先跑通，再解释，再重写

这份手册围绕你已经拿到的 A 股隔夜反转项目展开。目标不是背框架名字，而是让你能沿着一次研究请求解释：输入如何校验，谁调用工具，结果如何保存，页面如何拿到报告。

推荐顺序

第一遍读 02-06，建立概念并运行项目。第二遍边看源码边读 07-24，追踪一次请求。第三遍完成 26 的练习，再用 28-29 的面试问题检查自己是否真的会讲。

学习阶段	重点	达到什么程度
先跑通	安装、离线流程、页面	能提交实验并找到报告
看懂核心	输入模型、工具、状态图	能解释每一层输入输出
看懂服务	API、线程池、数据库、前端	能从点击追踪到报告
看懂 Agent	消息、工具调用、循环、MCP	能分清模型请求与程序执行
自己做	新增工具、接入节点、验证	能写出自己的小扩展

代码怎么阅读

手册中的“项目节选”来自实际代码，可能缩短变量名或省略错误处理；“教学示例”可另存为练习文件，不能当作完整生产实现。完整实现以相应源码路径为准。

命令默认在项目根目录的 PowerShell 中执行。后文使用的相对源码路径都指向 ai-hedge-fund-cn。Python 解释器使用独立虚拟环境，不需要先运行 Activate.ps1。

本次学习的完成标准

你能从零写一个纯 Python 工具，把它加入 LangGraph；通过 FastAPI 提交后台实验；再用 MCP 客户端发现并调用同一个工具。最后能解释失败处理和项目边界，而不把人工数据的收益当成策略证明。

LESSON 02 / BUILD IT STEP BY STEP

四个名字分别负责什么

LangChain 是构建模型应用的工具生态；LangGraph 管理状态和执行流程；FastAPI 提供 HTTP 服务；MCP 是模型应用与外部工具沟通的标准协议。它们可以配合使用，但不是同一种东西。

组件	解决的问题	本项目如何使用
LangChain	模型接口、消息、工具等抽象	原上游有模型集成；新增模块主要通过 LangGraph 依赖带入 langchain-core
LangGraph	节点、状态、分支和循环	StateGraph 编排必跑研究与可选模型工具循环
FastAPI	接收请求、校验参数、返回 JSON	提供实验、数据、因子、报告接口
Uvicorn	启动并运行 ASGI 应用	加载 api.main:app，监听本机端口
MCP	发现和调用外部工具	FastMCP 通过 stdio 暴露 6 个研究工具
httpx	发送 HTTP 请求	模型调用和部分测试使用它

一个重要纠正

当前新增模块没有直接导入 ChatOpenAI，也没有直接使用 LangChain 的 create_agent 或 LCEL 链。research_agent/llm.py 用 httpx 调用兼容 chat-completions 的服务。不能在面试中说“我用 LangChain Agent 完成了所有编排”。准确说法是“主要用 LangGraph，模型客户端采用直接 HTTP 协议”。

你需要掌握的区别

模型客户端负责“把消息发送给模型”。LangGraph 负责“下一步执行哪个函数”。FastAPI 负责“应用如何对外接收请求”。MCP 负责“外部 Agent 如何知道有这个工具，并以标准消息调用”。框架不会自动替你验证数据、修正回测或保存实验。

第 15 节会给出一个基于现有 langchain-core 的工具封装练习，让你实际体验 LangChain 的抽象，但它属于扩展练习，不是当前主流程已经具备的能力。

LESSON 03 / BUILD IT STEP BY STEP

整体架构：三个入口，一个计算核心

本项目先有可独立运行的研究核心，再增加工具层、编排层与服务层。这样不用在 HTTP 路由或模型提示词里重新实现收益计算。

```text
浏览器 -> FastAPI -> ResearchService -> LangGraph
                                          |
命令行 ------------------------------> |
                                          v
                                    ResearchTools
                                          |
外部 MCP 客户端 -> MCP Server ----------> |
                                          v
                          CSV 规范 / 因子 / 逐日回测
```

图中最容易理解错的一点

LangGraph 目前直接调用进程内的 ResearchTools，并没有通过 MCP 去调用另一个服务。MCP 是额外的标准工具入口。两者复用同一套业务函数，不能说“图中的每次工具调用都是 MCP 调用”。

为什么先写纯 Python 核心

如果收益计算绑在 API 路由里，你在单元测试、命令行和 MCP 中会重复实现。如果让模型计算收益，输出又可能不稳定。本项目把数据处理和回测放到 cn_research，让模型只读证据、提出附加查询并生成说明。

设计边界

当前是一个研究助手流程，不是多个自主 Agent 辩论的系统。没有 RAG、向量库、真实下单、分布式队列或持久 LangGraph checkpoint。SQLite 保存实验结果和事件，但不等于可以从任意图节点恢复执行。

LESSON 04 / BUILD IT STEP BY STEP

源码地图：应该按什么顺序打开文件

不要从一千行的上游项目开始。先打开新增目录，按“输入 -> 工具 -> 图 -> 服务 -> 协议”阅读。

文件或目录	你要找到的东西
cn_research/data.py	CSV 规范、字段和基础检查
cn_research/backtest.py	因子、逐日现金账本、T+1 和费用
research_agent/schemas.py	Pydantic 输入类型与参数边界
research_agent/tools.py	统一工具注册表、调用入口
research_agent/graph.py	state、节点、边和循环路由
research_agent/llm.py	模型协议、工具调用返回值校验
research_agent/service.py	后台执行、数据副本、报告
research_agent/storage.py	SQLite 任务与事件
api/main.py	HTTP 路由、服务生命周期、静态页面
frontend/app.js	提交、轮询、显示指标与报告
mcp_server/server.py	MCP 工具注册与 stdio 启动
tests/	核心回测与第二阶段集成测试

阅读技巧

先找公开入口：run_backtest、ResearchTools.call、run_research、ResearchService.submit、create_app、create_server。每读一个函数，记录四项：它接收什么、返回什么、可能失败在哪里、是否改变状态。

上游与新增模块的关系

保留的 hedge_fund 是原开源代码。新增 A 股模块使用自己的事件驱动研究引擎，并没有把整个美股主交易引擎完全改造成 A 股。面试时把自己的新增目录、修正内容和上游保留部分说清楚，二开才可信。

LESSON 05 / BUILD IT STEP BY STEP

第一步：安装依赖并跑通离线流程

需要 Python 3.11 或以上。先进入项目目录，再使用一键启动。你的本机目录是 C:/Users/qqq/OneDrive/文档/ChatGPT/建模/ai-hedge-fund-cn。

```powershell
cd 'C:/Users/qqq/OneDrive/文档/ChatGPT/建模/ai-hedge-fund-cn'
python start_stage2.py
```

启动脚本实际做了什么

start_stage2.py 检查 Python 版本和端口；若不存在 .venv-stage2 就创建虚拟环境；首次安装 requirements-stage2-lock.txt；最后执行虚拟环境中的 Uvicorn。后续普通启动不会再次安装全部依赖。需要补装时使用 --install。

```powershell
# 在项目根目录，手动验证同一个解释器
.venv-stage2/Scripts/python.exe -m pip check
.venv-stage2/Scripts/python.exe -m research_agent
.venv-stage2/Scripts/python.exe -m pytest tests -q
```

requirements-stage2.txt 记录直接依赖和允许范围；requirements-stage2-lock.txt 是验证环境的固定版本快照。原上游 Poetry 环境与新增环境分开，避免旧 LangChain 依赖冲突。

正确的预期结果

页面地址是 http://127.0.0.1:8000，接口文档是 /docs。离线 CLI 会打印实验 ID 和报告路径。已验证的人工样例有 4 笔退出交易，净收益约 -4.30%。这个值用于检查演示链路，不用于判断因子有效。

如果端口已有本项目服务，不必再启动第二个进程；直接打开页面。需要独立启动时用 python start_stage2.py --port 8001。不要同时启动多个写同一任务库的服务。

LESSON 06 / BUILD IT STEP BY STEP

第二步：先理解因子和回测核心

Agent 项目不能只展示一段漂亮文字。它需要能被验证的计算核心。这里的工具围绕一个明确的隔夜反转因子建立。

```python
# 定义说明，不是完整回测
overnight_return = open_t / close_previous_session - 1
factor = -overnight_return
```

若昨收 10 元、今开 9 元，隔夜收益为 -10%，反转因子为 +10%。每天把股票按 factor 从大到小排序，入选 ceil(N * quantile) 只。默认 quantile=0.2，即最高 20% 分组；同因子按股票代码确定顺序。

时间线

```text
T 日收盘：形成信号，使用已知数据
T+1 开盘：按信号尝试买入
T+2 收盘：第一次尝试卖出
后续收盘：若无法卖出，继续持仓和重试
```

信号虽然在当天开盘后就可计算，当前设计到收盘形成，次日才执行。这样不会使用开盘后的信息又假设同一开盘价成交；买入当天不卖出，满足本模型的 T+1 约束。

核心函数的输出

run_backtest 返回 config、metrics、signals、fills、trades、equity_curve、open_positions 和 skipped_orders。metrics 是汇总，fills 是实际成交账本，open_positions 则防止期末未退出仓位从结果中消失。

先能运行核心，才接框架

```python
from cn_research.data import load_csv
from cn_research.backtest import run_backtest

bars = load_csv('data/sample_ashare.csv')
result = run_backtest(bars, quantile=0.2)
print(result['metrics']['total_return'])
```

LESSON 07 / BUILD IT STEP BY STEP

第三步：先给工具定义输入契约

“输入契约”就是明确函数接受哪些字段、类型和取值。Pydantic 在运行时做校验；TypedDict 主要帮助静态类型分析，不能替代输入检查。

PYTHON

```
# 教学缩写，完整实现见 schemas.py
from pydantic import BaseModel, ConfigDict, Field

class BacktestInput(BaseModel):
    model_config = ConfigDict(
        extra='forbid', allow_inf_nan=False
    )
    dataset_id: str = 'sample_ashare'
    quantile: float = Field(default=0.2, gt=0, le=0.5)
```

每一行的作用

BaseModel 定义可构造的参数对象。Field 指定默认值和边界。extra='forbid' 让未知字段报错，避免拼错的字段悄悄被忽略。allow_inf_nan=False 拒绝 NaN 和无穷值。实际 DatasetInput 还限制了 dataset_id 的字符和长度。

PYTHON

```
from research_agent.schemas import BacktestInput

args = BacktestInput(quantile=0.2)
print(args.model_dump())
print(BacktestInput.model_json_schema()['properties'])
# BacktestInput(quantile=0.8) 会抛出 ValidationError
```

model_dump() 把对象变成普通字典；model_validate(dict) 把外部字典转成受校验对象；model_json_schema() 给出机器可读的参数规范。这三种转换分别用于日志、工具执行和模型工具说明。

为什么模型输出也要校验

模型可能给出不存在的工具、错误类型或过长参数。不能因为它“像 JSON”就信任它。研究参数、HTTP 入参和模型工具参数最终都需要落到同一套明确约束上。

LESSON 08 / BUILD IT STEP BY STEP

第四步：把计算封装成共享工具

一个工具本质上仍是 Python 函数。区别在于它有名字、说明、输入规范、输出结构和受控调用方式。research_agent/tools.py 把这些信息集中登记。

PYTHON

```
# 项目节选
self.specs = {
    'run_backtest': (
        BacktestInput,
        self.run_backtest,
        'Run deterministic T+1 backtest'
    )
}

def call(self, name, arguments):
    if name not in self.specs:
        raise ValueError(f'unknown tool: {name}')
    schema, fn, _ = self.specs[name]
    return fn(schema.model_validate(arguments))
```

调用过程

call 收到工具名字和字典。先检查名字是否在注册表；拿到对应输入类和函数；验证字典；然后把验证过的对象传给函数。这里执行真实计算的是 Python，不是模型。

PYTHON

```
from research_agent.tools import ResearchTools

tools = ResearchTools()
evidence = tools.call('run_backtest', {
    'dataset_id': 'sample_ashare', 'quantile': 0.2
})
print(evidence['metrics']['n_trades'])
```

完整结果与摘要为什么分开

full_backtest 返回成交和逐日净值等完整数据，方便报告和页面。run_backtest 工具返回配置、指标和数量摘要，避免把海量交易记录塞进模型上下文。当前项目没有精细 token 预算，超大股票池的摘要仍需进一步控制。

LESSON 09 / BUILD IT STEP BY STEP

第五步：LangGraph 的 state 和 node

StateGraph 的三个基本概念是状态、节点和边。状态是当前研究的上下文；节点是一个处理函数；边规定处理顺序。节点通常只返回自己修改的部分，图把这部分合并进 state。

PYTHON

```python
# 项目节选的简化版本
from typing import TypedDict

class ResearchState(TypedDict, total=False):
    request: dict
    quality: dict
    backtest: dict
    summary: str

def validate_data(state):
    args = DatasetInput(
        dataset_id=state['request']['dataset_id']
    )
    result = tools.validate_dataset(args)
    return {'quality': result}
```

不要误解 return

return {'quality': result} 不表示清空 request。默认情况下它更新 quality 字段，其他 state 字段仍在。当前图没有给 messages 配置追加 reducer，所以代码显式用旧消息列表加新消息，再把完整列表返回。

纯函数与副作用

validate_data 的核心工作是读取数据并返回摘要。emit(...) 则是记录事件的副作用。在图里保留这两个职责的区别，有助于测试：同样输入应得到相同证据，事件用于观察执行进度。

小练习

打开 graph.py，对照每个节点写出它读哪些 state 字段、写哪些字段。validate 写 quality，factor 写 factor，backtest 写 backtest，costs 写 cost_comparison 和 messages，finish 写 summary。先掌握数据流，再学习循环。

LESSON 10 / BUILD IT STEP BY STEP

第六步：把节点连成可执行的图

研究基本证据必须先完成，不能由模型决定“是否需要回测”。因此 graph.py 固定连上数据检查、因子快照、回测和成本对照，然后才进入可选模型分析。

PYTHON

```
# 教学缩写：展示注册和连边
from langgraph.graph import StateGraph, START, END

graph = StateGraph(ResearchState)
graph.add_node('validate_data', validate_data)
graph.add_node('run_backtest', backtest_node)
graph.add_edge(START, 'validate_data')
graph.add_edge('validate_data', 'run_backtest')
graph.add_edge('run_backtest', END)
app = graph.compile()
state = app.invoke({'request': request_dict})
```

add_node 登记处理函数；add_edge 声明固定顺序；compile 生成能运行的图；invoke 提供初始 state 并执行。名字只是图中的标识，不要求和 Python 函数名完全相同。

运行项目中的完整图

PYTHON

```
from research_agent.graph import run_research
from research_agent.schemas import ExperimentInput
from research_agent.tools import ResearchTools

state = run_research(
    ExperimentInput(mode='offline'), ResearchTools()
)
print(state['summary'])
```

离线模式依然经过 LangGraph，但 analyst 不调用模型，finish 按真实指标生成规则结论。使用图不意味着已经有自主 Agent；执行中是否存在模型选择工具和分支，才决定你能描述到什么程度。

LESSON 11 / BUILD IT STEP BY STEP

第七步：给图增加条件分支和循环

固定工作流只能按顺序执行。模型模式需要“看证据 -> 请求工具 -> 得到结果 -> 再分析”。条件边根据 state 决定走向，因此可以实现循环。

PYTHON

```
# 项目节选
def route(state):
    if state['request']['mode'] == 'offline':
        return 'finish'
    if state.get('llm_error'):
        return 'finish'
    if state['messages'][-1].get('tool_calls'):
        return 'tools' if state['rounds'] <= 3 else 'finish'
    return 'finish'

graph.add_conditional_edges('analyst', route, {
    'tools': 'tools', 'finish': 'finish'
})
graph.add_edge('tools', 'analyst')
```

rounds 为什么最大会是 4

第一次模型响应 rounds=1，可以请求工具。三轮工具之后，可以再请求一次模型最终回答，此时 rounds=4。如果它仍请求工具，就结束并回退规则结论。因此上限是 3 轮工具执行、最多 4 次模型请求，而不是“模型总共只调用 3 次”。

两种上限解决不同问题

业务规则用 rounds 限制模型请求。图的 recursion_limit=20 是额外的图执行保护，不是模型 token 限额，也不是断点恢复机制。实际每次 HTTP 请求还设置了 45 秒 timeout。

应该会回答的问题

为什么不能 while True 无限循环？工具可能重复请求，模型也可能一直不结束。上限让成本和等待时间有边界，并保留已经完成的确定性证据。

LESSON 12 / BUILD IT STEP BY STEP

第八步：模型请求里到底发送了什么

research_agent/llm.py 使用 httpx。当前模型服务需支持 chat-completions 的工具协议。这个客户端不执行 Python 函数，只发送消息并接收模型给出的内容或调用建议。

PYTHON

```
# 教学缩写，省略鉴权和错误处理
response = client.post(base + '/chat/completions', json={
    'model': model_name,
    'messages': messages,
    'tools': tools.llm_definitions(),
    'tool_choice': 'auto',
    'temperature': 0,
    'max_tokens': 1200
})
message = response.json()['choices'][0]['message']
```

三种角色

system 消息描述研究角色和行为约束；user 消息包含研究问题与工具证据；assistant 消息是模型回答或工具调用请求。程序执行工具后，会追加 role='tool' 的消息，把工具结果交回模型。

工具说明从哪里来

llm_definitions() 为注册表中的每个工具生成名字、description 和 parameters。parameters 来自输入类的 model_json_schema()。模型看到的是函数说明和参数结构，不是完整函数代码。

模型配置练习

复制 .stage2.env.example 为 .stage2.env，填写 LLM_BASE_URL、LLM_MODEL、LLM_API_KEY，重启服务。BASE_URL 是不含 /chat/completions 的基础地址。服务必须真正支持 tools 协议；仅支持普通文本对话的端点不够。

目前只验证了模拟模型与 HTTP stub，没有接入真实密钥验证生成质量。实际使用时，研究问题和证据会发送到你配置的模型服务。

LESSON 13 / BUILD IT STEP BY STEP

第九步：看懂一次 tool_call 返回值

模型“调用工具”时，一般没有自己运行你的 Python。它先返回一个描述：要调用哪个工具、传什么参数。程序收到以后才决定是否允许，并执行真实函数。

JSON

```
{
  "role": "assistant",
  "tool_calls": [{
    "id": "call_1", "type": "function",
    "function": {
      "name": "compare_costs",
      "arguments": "{\"quantile\":0.2}"
    }
  }]
}
```

arguments 为什么还要 json.loads

这里 arguments 是包含 JSON 的字符串，不是 Python 字典。需要 json.loads 解码，再用输入模型验证。直接把字符串用 eval 执行是错误做法：这会把模型输出当代码运行。

PYTHON

```
# graph.py 的工具执行逻辑节选
name = call['function']['name']
arguments = json.loads(call['function']['arguments'])
arguments['dataset_id'] = state['request']['dataset_id']
output = tools.call(name, arguments)
messages.append({
    'role': 'tool', 'tool_call_id': call['id'],
    'content': json.dumps(output, ensure_ascii=False)
})
```

tool_call_id 把工具结果和之前的调用配对。dataset_id 被强制绑定到当前实验副本，模型不能要求读取另一份文件。tools.call 做名字和参数校验，输出才会成为新的模型证据。

不同分位查询怎么处理

模型可追加不同参数的描述性比较，但不能改掉本次基础回测的 backtest state。原表单参数仍是权威结果；附加查询只提供更多分析证据。

LESSON 14 / BUILD IT STEP BY STEP

第十步：错误处理和事实边界

模型写出的结论可能错误，即使工具真实返回了指标，也不能保证它逐字准确引用。本项目通过结构分离保留权威数据：backtest 由代码产生，summary 是分析文字，两者不能相互覆盖。

错误或风险	当前行为	你应理解的边界
数据结构错误	任务 failed，记录原因	不能当作没有信号
不存在的工具	返回工具错误消息	白名单执行，不执行任意代码
模型 401 或网络失败	保留回测，规则分析回退	请求失败不等于研究核心失败
反复请求工具	达到上限后结束	成本和等待有边界
工具参数错	Pydantic 校验失败	模型建议不是可信输入
模型编造数字	结构化结果仍保留	尚无自动事实核对，需进一步评估

模型客户端的校验

AssistantMessage 校验 role、文字长度和 tool_calls 数量；每轮最多 3 个调用，每个调用有 id、函数名和参数字符串。HTTP 错误不把服务端响应正文写进报告，以免泄露敏感信息。

本项目做到哪里

提示词要求只引用证据，代码限制工具权限，结果区保留原始计算。但没有独立“事实核查 Agent”、完整 Prompt Injection 防护或生成质量评测集。提示词规则是约束意图，不是可证明的安全保证。

学习任务

先不接真实模型，读 tests/test_stage2.py 中 FailingModel、RepeatingModel、ToolCallingModel 三个例子。它们分别验证失败回退、循环终止和工具执行。不要把这些模拟测试说成真实模型效果测试。

LESSON 15 / BUILD IT STEP BY STEP

可选练习：用 LangChain 包装同一个工具

你问到 LangChain，但主流程没有直接使用它的 Agent 封装。可以做一个不需要 API Key 的练习：把现有 Python 函数转成 LangChain StructuredTool。langchain-core 已由 LangGraph 依赖安装。

PYTHON

```python
# 教学示例：另存为 learn_langchain.py
from langchain_core.tools import StructuredTool
from research_agent.schemas import BacktestInput
from research_agent.tools import ResearchTools

tools = ResearchTools()

def backtest_tool(**kwargs) -> dict:
    return tools.call('run_backtest', kwargs)

lc_tool = StructuredTool.from_function(
    func=backtest_tool,
    name='run_backtest',
    description='Run the deterministic A-share backtest.',
    args_schema=BacktestInput,
)
result = lc_tool.invoke({'dataset_id': 'sample_ashare'})
print(result['metrics']['n_trades'])
```

POWERSHELL

```powershell
.venv-stage2/Scripts/python.exe learn_langchain.py
```

这一步发生了什么

同一函数被包装成标准工具对象。名字、说明、输入 schema 和 invoke 接口被统一了。业务计算没变，也没有多出一个模型调用；不应该因此把它称为完整 Agent。

如果以后替换模型客户端

可以使用支持所选服务的 LangChain 模型适配器和 bind_tools，把框架消息对象转换成图当前需要的消息结构。要验证工具返回、超时、重试和错误类型；不是把 import 改一下就完成迁移。当前项目不要求安装 langchain-openai，本练习也不使用 ChatOpenAI。

LESSON 16 / BUILD IT STEP BY STEP

第十一步：FastAPI 把 Python 函数变成 HTTP 接口

HTTP 接口把外部请求映射到 Python 函数。最容易起步的方式，是先写一个只返回健康状态的小服务，再接入现有工具。

PYTHON

```python
# 教学示例：另存为 learn_api.py
from fastapi import FastAPI
from research_agent.schemas import BacktestInput
from research_agent.tools import ResearchTools

app = FastAPI()
tools = ResearchTools()

@app.get('/health')
def health():
    return {'status': 'ok'}

@app.post('/backtest')
def backtest(payload: BacktestInput):
    return tools.run_backtest(payload)
```

POWERSHELL

```powershell
.venv-stage2/Scripts/python.exe -m uvicorn learn_api:app --port 8001
```

逐行解释

FastAPI() 创建应用；@app.get 把 URL 和函数绑定；payload: BacktestInput 告诉框架把 JSON 请求解析成受校验对象；返回字典会转成 JSON。/docs 自动根据类型生成交互文档。

Uvicorn 的 learn_api:app 表示“导入 learn_api 模块，使用里面名为 app 的对象”。项目实际启动用 api.main:app。端口 8001 让练习服务与现有页面分开。

这个教学接口是等待计算完成后返回结果，不是后台任务版本。打开 http://127.0.0.1:8001/docs，在 POST /backtest 中点 Try it out，传入 {"dataset_id":"sample_ashare"} 即可试跑。下一节说明项目为什么先返回实验 ID。

LESSON 17 / BUILD IT STEP BY STEP

第十二步：长任务先返回 202 和实验 ID

回测或模型调用可能较慢。若一次请求一直等到最后，浏览器容易超时。api/main.py 的 /api/experiments 只接收并提交任务，立即返回 202；浏览器随后查询状态。

PYTHON

```
# 项目节选，省略错误映射
@app.post('/api/experiments', status_code=202)
def submit(req: Request, payload: ExperimentInput):
    ident = req.app.state.service.submit(payload)
    return {
        'id': ident, 'status': 'queued',
        'url': f'/api/experiments/{ident}'
    }
```

为什么用 ResearchService

HTTP 层只处理请求和响应。ResearchService 负责排队、保存输入副本、执行图和写报告。这样图不用知道自己来自网页还是命令行，服务也不用把模型逻辑写进路由。

HTTP 状态码要能解释

200 表示查询成功；202 表示已接收任务但未完成；404 表示数据或实验不存在；422 表示参数不符合规范；429 表示队列容量已满；409 表示报告尚未准备好。任务执行失败则保存为 failed，在实验查询中返回明确原因。

同步路由不等于没有并发

当前大部分路由是 def，FastAPI 会在线程池中运行同步路径操作。实际研究任务又被提交到自己的 ThreadPoolExecutor。不要把代码说成“全部 async/await 异步计算”。计算并发与 HTTP 框架的异步 I/O 是不同层次。

LESSON 18 / BUILD IT STEP BY STEP

第十三步：线程池、队列容量和生命周期

ResearchService 使用两个工作线程，并用 BoundedSemaphore 限制最多 8 个在途任务。这里的 8 包括正在执行和排队的任务，不是“8 个线程同时跑”。

PYTHON

```
# 项目节选
self.pool = ThreadPoolExecutor(max_workers=2)
self.slots = BoundedSemaphore(8)

if not self.slots.acquire(blocking=False):
    raise RuntimeError('research queue full; retry later')
ident = self.store.create(request.model_dump(mode='json'))
self.pool.submit(self.execute, ident, request)
```

为什么 finally 很重要

execute 无论成功或失败，都在 finally 释放 slots。否则一次失败会永久占住容量，运行一段时间后所有请求都返回队列满。submit 本身失败时也需要释放已取得的名额。

启动和关闭

api/main.py 用 lifespan 在启动时创建 service，关闭时调用 pool.shutdown(wait=True)。启动还把上一次中断的 queued/running 任务标记 failed。实验历史能保留，但需要重新提交，并不是恢复到原来的图节点。

什么时候该换方案

线程池适合本地 MVP 和 I/O 等待。多个进程、分布式工作器、严格任务超时或 CPU 密集回测需要独立任务队列与更完整的部署设计。当前要用单个 API 进程，不能开多个 worker 写同一任务库。

LESSON 19 / BUILD IT STEP BY STEP

第十四步：保存实验，而不仅是保存一段文字

一个可以复现的研究实验应该保留输入、参数、版本、过程和结果。只保存 summary，无法解释模型引用的是哪份数据，也无法复查成交。

```text
outputs/stage2/
  experiments.sqlite3
  <experiment-id>/
    input/sample_ashare.csv
    research_report.md
    research_report.json
```

执行流程

service.execute 先把状态改成 running；复制 CSV 到本实验目录；用副本创建 ResearchTools；运行图并把 emit 事件写入 SQLite；生成报告；最后保存 completed。如果数据或执行失败，就保存 failed 和原因。

数据库中有什么

jobs 表保存 id、status、时间、request、result 和 error。events 表保存事件顺序、所属实验、时间和 payload。SQL 使用参数占位符绑定值，不拼接外部输入到 SQL 文本。

SHA256 能证明什么

哈希标识实际使用的文件内容，帮助你判断两次实验是否读取了同一份数据。它不能证明数据来源正确、没有幸存者偏差，或因子具有统计显著性。副本也不是加密存储。

JobStore 不是 LangGraph checkpoint

当前数据库记录结果和节点事件，没有为图配置 checkpointer。程序中断后不能自动从 compare_costs 接着执行。面试中必须把“实验可追踪”与“执行可恢复”分开讲。

LESSON 20 / BUILD IT STEP BY STEP

第十五步：前端如何提交实验

前端是普通 HTML、CSS 和 JavaScript，没有 React，也不需要 Node/npm。它和 API 由同一个 FastAPI 服务提供，浏览器直接请求相对路径。

JAVASCRIPT

```
// 教学缩写，省略表单读取和错误处理
const response = await fetch('/api/experiments', {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({
    dataset_id: 'sample_ashare',
    quantile: 0.2,
    mode: 'offline'
  })
});
const job = await response.json();
poll(job.id);
```

JSON 不是 Python 字典

浏览器把 JavaScript 对象序列化成 JSON 文本，FastAPI 再解析并校验为 ExperimentInput。数值输入从 FormData 读出通常是字符串，因此 app.js 对资金、成本和 quantile 使用 Number(...) 转换。

页面文件怎样送到浏览器

api/main.py 的 / 返回 index.html，/static 挂载 frontend 目录。HTML 加载 style.css 和 app.js。浏览器提交 POST，不会直接调用 Python 文件。

同源与 CORS

页面和 API 同协议、域名和端口，因此当前不需要跨域配置。如果以后用独立前端开发服务，例如 5173 端口，就要明确配置允许的来源；不能因为本地演示方便就把所有来源都当作生产配置。

LESSON 21 / BUILD IT STEP BY STEP

第十六步：前端轮询、展示和错误恢复

收到实验 ID 后，页面通过 GET /api/experiments/{id} 读取状态。app.js 大约每 650 毫秒再查询一次，直到 completed 或 failed。这里使用轮询，不是 WebSocket 或 SSE 实时推送。

JAVASCRIPT

```
// 教学缩写
async function poll(id) {
  const response = await fetch(`/api/experiments/${id}`);
  const job = await response.json();
  if (job.status === 'completed') {
    render(job.result, id);
  } else if (job.status === 'failed') {
    showError(job.error);
  } else {
    setTimeout(() => poll(id), 650);
  }
}
```

前端从哪里拿指标

result.backtest.metrics 提供收益、回撤和成交数；equity_curve 提供每日净值；trades 和 open_positions 提供退出及期末持仓；quality.warnings 显示数据限制；summary 显示研究分析。模型文字不会替换指标卡。

为什么使用 textContent

研究问题、模型回答和错误文字通过 textContent 显示，避免把不受信任的文本当 HTML 执行。图表是代码生成的 SVG，坐标来自计算结果。当前没有使用外部图表库。

刷新以后怎么办

提交后把实验 ID 写到 URL hash。刷新时读取 ID 并重新查询已保存结果。最近实验来自 SQLite，页面不需要自己保存整个报告。若报告完成，可通过接口下载 Markdown 或读取完整 JSON。

LESSON 22 / BUILD IT STEP BY STEP

第十七步：MCP 不是另一种 FastAPI

FastAPI 通常向应用提供 HTTP 端点；MCP 定义工具发现与调用的协议。当前 MCP Server 使用 stdio，客户端启动一个子进程，通过标准输入和输出传输协议消息，不监听 8000 端口。

PYTHON

```
# 项目节选
from typing import Any
from mcp.server.fastmcp import FastMCP

server = FastMCP('A-share Research Tools')

@server.tool()
def validate_dataset(
    dataset_id: str = 'sample_ashare'
) -> dict[str, Any]:
    return tools.call('validate_dataset', {
        'dataset_id': dataset_id
    })

server.run(transport='stdio')
```

装饰器注册了什么

@server.tool() 注册函数名、说明、参数类型和返回类型。客户端可以 list_tools 发现这些信息，再 call_tool 调用。FastMCP 是 Python MCP SDK 的封装，与 FastAPI 是不同库，名字相似不代表是同一个服务。

为什么返回 dict[str, Any]

明确的返回注解有助于 SDK 生成结构化输出。实际测试发现 bare dict 未产生预期 structuredContent，改成参数化注解后验证通过。模型应用需要机器可读结果，而不只是看起来像 JSON 的文字。

六个工具

list_datasets、validate_dataset、compute_factor、run_backtest、compare_costs、parameter_sensitivity。它们都是只读研究工具，不包含真实下单或任意文件操作。

LESSON 23 / BUILD IT STEP BY STEP

第十八步：从客户端真正调用 MCP

不要只看 Server 能启动。需要用 MCP Client 完成 initialize、list_tools、call_tool，确认双方真的能通信。项目测试用标准 Python SDK 启动 stdio 子进程。

PYTHON

```python
# 可运行教学示例：另存为 learn_mcp.py
import asyncio, sys
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def main():
    params = StdioServerParameters(
        command=sys.executable,
        args=['-m', 'mcp_server.server']
    )
    async with stdio_client(params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            names = await session.list_tools()
            print([t.name for t in names.tools])
            result = await session.call_tool(
                'run_backtest',
                {'dataset_id': 'sample_ashare'}
            )
            print(result.structuredContent['metrics'])

asyncio.run(main())
```

POWERSHELL

```powershell
.venv-stage2/Scripts/python.exe learn_mcp.py
```

必须用虚拟环境解释器在项目根目录运行，这样子进程也能导入项目模块。stdio_client 管理子进程生命周期；ClientSession 管理协议会话。客户端自己打印结果没问题，但 Server 的 stdout 不能混入调试 print，否则破坏协议。

外部应用如何配置

mcp_server/client.example.json 中指定绝对 Python 路径、-m mcp_server.server 和项目 PYTHONPATH。按客户端支持的格式填写，不是把 JSON 粘到网页里。调用 MCP 不需要先启动 FastAPI。

LESSON 24 / BUILD IT STEP BY STEP

一次点击的完整路径：把各层串起来

现在回到页面的“开始研究”。如果能逐步讲清这条路径，你就不只是会运行项目，而是在理解系统。

```text
1. 表单 -> JavaScript 对象 -> JSON
2. POST /api/experiments -> Pydantic 校验
3. ResearchService.submit -> SQLite queued
4. ThreadPoolExecutor -> execute -> running
5. CSV 副本 -> ResearchTools -> LangGraph
6. validate / factor / backtest / compare_costs
7. offline: 规则报告
   llm: 模型 -> tool_calls -> 工具 -> 再分析
8. Markdown + JSON + SQLite completed
9. 浏览器 GET 轮询 -> 展示 -> 下载
```

这条链路中的输入输出

用户输入不是随意执行的指令，而是受校验的研究请求。图中的证据来自计算工具，工具只读取当前实验副本。模型接收摘要，可以追加查询。最终返回的 result 包含基础回测与分析来源，前端分别显示。

MCP 在哪条链路

外部应用的 MCP 请求 -> Server 工具函数 -> ResearchTools.call -> 计算核心 -> structuredContent。它没有默认创建后台实验，也不经过前端。要把 MCP 用作 LangGraph 的远程工具层，是后续另一项改造，需要客户端连接和结果映射。

练习：在图上标出失败点

参数错在 API 校验阶段；数据缺失可能在提交或读数据阶段；模型错误在 analyst；工具错误会返回工具消息；报告写入错误会使任务失败。说明你知道不同失败发生在不同边界，而不是统一解释为“模型不行”。

LESSON 25 / BUILD IT STEP BY STEP

验证：怎样证明自己写的东西能用

项目已验证 29 个测试。学习时重点不是背数量，而是读每个测试对应的风险。只测试“函数返回一个字典”，不能证明回测正确或服务可用。

测试类别	验证的事实
因子	昨收和今开正确对齐，符号正确
未来信息隔离	改变未来行情不影响过去买入和净值
交易账本	T+1、整手、现金、费用、受阻持仓
数据与参数	重复、非有限值、错误字段被拒绝
图与模型	必跑节点、工具循环、上限、失败回退
API	202 提交、状态完成、报告、持久读取
MCP	真实子进程握手、发现、调用、错误

项目中的回归测试思想

第一阶段曾使用未来退出日是否可成交来过滤买入名单，并对可退出股票重新等权。这是未来信息泄漏。第二阶段逐日执行，把无法卖出的仓位留在账上。测试删除未来退出行情后，过去的信号和买入必须完全相同。

```powershell
.venv-stage2/Scripts/python.exe -m pytest tests -q
.venv-stage2/Scripts/python.exe -m pip check
```

应该如实描述的限制

日线成交仍是近似，开盘成交结合全天成交量判断属于简化，不能据此宣称盘中实盘级无偏模拟。未处理完整公司行动、退市样本、官方日历和历史税率变化。真实模型效果、真实市场大样本和上游全部测试也未在本阶段验证。

LESSON 26 / BUILD IT STEP BY STEP

练习：新增一个你自己的波动率工具

建议先完成一个只读工具：用历史收盘收益计算波动率。它会经过数据、输入 schema、工具注册、模型说明与 MCP 五个环节，能证明你理解了扩展路径。

实现要求

输入包含 dataset_id、code、as_of 和 window。只取 date <= as_of 的该股票，价格来自已有 frame。window 至少 5；取至少 window+1 个收盘价；若历史不够应明确报错；输出实际起止日、收益观测数和年化波动率。

PYTHON

```
# 计算核心的教学片段
history = df[(df.code == code) & (df.date <= as_of)]
history = history.sort_values('date').tail(window + 1)
returns = history.close.pct_change().dropna()
if len(returns) < window:
    raise ValueError('insufficient history')
volatility = float(returns.std(ddof=1) * (252 ** 0.5))
```

接入顺序

先在 schemas.py 写 VolatilityInput，再给 ResearchTools 增加方法和 specs 项；用 tools.call 手动验证。注册表会使 llm_definitions 自动包含它。再在 MCP 中显式写 @server.tool 包装，因为 MCP 不会自动扫描 specs。API 是否增加新路由取决于你希望页面如何访问。

三个必须做的检查

改变 as_of 之后的数据，输出不变；股票不存在或历史不足时，清楚报错；工具返回的数值与手动计算一致。样例仅 10 个日期，不适合 window=20，用 window=5 或增加适合的测试数据。

最后在博客写输入、数学定义、边界和接入位置，并保留自己的实现提交。只增加一个 Prompt，不算完成这个工程练习。

LESSON 27 / BUILD IT STEP BY STEP

五天学习计划：每天有一个可检查的产物

下面是学习节奏，不是保证五天达到生产开发水平。每天可以用 2-4 小时动手，遇到不懂的 Python 语法要停下来实验。

日期	读哪些章节	必须做出的东西
第 1 天	02-08	跑通核心，手算因子，写一个参数校验例子
第 2 天	09-15	写最小两节点图，解释工具循环，跑 LangChain 包装练习
第 3 天	16-21	在 8001 跑教学 API，用 /docs 提交，并读懂前端轮询
第 4 天	22-26	跑 MCP Client，新增波动率工具并验证 as_of
第 5 天	28-29	画完整调用链，录一次演示，练面试并写博客

每天结束的自测

第 1 天：我能说明为什么计算不能交给模型。第 2 天：我能把 tool_calls 变成真实 Python 函数执行。第 3 天：我能区分 HTTP 202 与实验 completed。第 4 天：我能说明 MCP 和进程内工具调用的差别。第 5 天：我能在不夸大能力的情况下说清贡献和限制。

博客建议按问题组织

写“为什么把计算与接口分开”“回测如何避免未来退出信息”“为什么增加任务 ID”“MCP 工具为什么要类型化”。每篇放一个最小代码、一个运行结果和一个踩坑。框架名列表不是技术博客的核心。

简历里的成果应该有证据

保留你真正完成的新增模块、测试、演示和复现命令。不要把预设教程复现、生成的说明书和原开源功能全部算作自己的独立设计。

LESSON 28 / BUILD IT STEP BY STEP

面试表达：两分钟讲清项目

可以按下面的顺序练习，但请在理解并完成练习后，用自己的语言表达。

参考讲法

“我在 ai-hedge-fund 的开源代码基础上新增了 A 股隔夜反转研究模块。先实现确定性因子和逐日现金账本，再把数据检查、因子快照、回测和成本比较封装成共享工具。

我用 LangGraph 管理研究 state、必跑节点以及可选模型的工具调用循环。模型只分析证据，不负责收益计算或真实下单。服务层使用 FastAPI，提交研究后返回实验 ID，通过线程池后台执行，并把参数、工具事件和报告保存到 SQLite 与文件。前端通过轮询展示净值和报告。

同一套工具也通过 MCP stdio 对外提供，外部 Agent 可以发现和调用工具。工程上增加了参数验证、输入副本和哈希、失败回退、循环上限与集成测试。我也修正了原型根据未来退出可成交性筛选买入的问题，用未来行情变更测试锁定过去决策。

目前是本地研究 MVP，默认人工数据，只验证工程链路，没有宣称策略有效、分布式恢复或实盘能力。”

不建议说的内容

不要说“做了完全自主多 Agent 对冲基金” “LangChain 自动解决了全部工具调用” “所有工具都通过 MCP” “29 个测试证明因子有效”。当前事实分别是单研究流程、直接 HTTP 模型客户端、图内直接调用工具、工程验证。

面试官真正关注什么

你为什么做这些选择；输入与输出边界是否清楚；异常如何处理；是否能写出工具和 API；能否解释一条失败案例；能否区分已有能力和后续计划。

LESSON 29 / BUILD IT STEP BY STEP

面试追问：先理解，再回答

1. 为什么用 LangGraph，不直接按顺序调用函数?

只有固定步骤时普通函数足够。这里还需要模型工具循环、状态更新和条件路由，用图可以明确表达这些控制关系。当前没有配置 checkpoint，因此不能顺带声称可断点恢复。

2. LangChain 和 LangGraph 是什么关系?

属于相邻生态，但职责不同。LangChain 提供模型与工具等应用抽象；LangGraph 管理有状态执行。当前新增模块主要用 LangGraph，模型请求由 httpx 发出，LangChain 工具包装只是练习扩展。

3. function calling 和 MCP 有什么不同?

function calling 是模型选择工具并给出参数的接口能力。MCP 是客户端与工具服务器发现、调用工具的协议。模型不一定直接理解 MCP，通常由客户端把 MCP 工具描述转成模型可用 schema，再执行并回传结果。

4. 为什么不能让模型直接操作文件或计算收益?

数学计算需要稳定和可核验；文件访问需要明确范围。本项目工具白名单和 dataset_id 限制执行边界。模型只提出受校验的调用建议，而非任意 Python 代码。

5. 为什么有了异步框架，还需要线程池?

API 请求生命周期与耗时研究任务不是一回事。当前通过任务 ID 与线程池分离完成时间；这只是单进程 MVP，不等于分布式队列。

6. 如果模型出错或者一直调用工具怎么办?

保留确定性证据，记录错误，有限轮数后规则回退。需分别评估工具正确性和模型结论质量。当前未做真实模型质量评测，不能用模拟测试替代。

7. 一个最值得改进的问题是什么?

可以选样本外研究、官方日历与公司行动、模型事实核对或任务队列。先说明它解决哪个具体失败场景，再提出实现与验证方法，避免只说“换更强模型”。

LESSON 30 / BUILD IT STEP BY STEP

排障与继续阅读

现象	先检查什么
ModuleNotFoundError	是否在项目根目录，是否用 .venv-stage2 解释器
端口占用	是否已有服务；用 --port 8001，勿同时写同一任务库
页面模型选项不可用	三项 LLM 环境变量是否完整，是否重启服务
模型 HTTP 401	密钥及服务权限；错误正文不会写进报告
模型 HTTP 404	BASE_URL 是否被重复拼接 /chat/completions
422	字段类型、取值和未知字段；对照 /docs schema
MCP 启动无页面	stdio Server 不是浏览器服务；应由 Client 连接
MCP structuredContent 为空	返回注解与 SDK 版本；查看错误或文本内容
任务一直等待或中断	查看实验 events；重启后中断任务失败，重新提交

本手册对应的本地来源

新增源码：research_agent/、api/、mcp_server/、frontend/、cn_research/。安装说明：README_STAGE2.md。验证说明：docs/stage2_validation.md。上游基线 commit：78b779c1389e2d1452dc29606d2c4126d859b964。

官方参考入口

LangGraph 文档：https://docs.langchain.com/oss/python/langgraph/overview

LangChain 文档：https://docs.langchain.com/oss/python/langchain/overview

FastAPI 教程：https://fastapi.tiangolo.com/tutorial/

MCP Python SDK：https://github.com/modelcontextprotocol/python-sdk

Pydantic 文档：https://docs.pydantic.dev/latest/

这些链接作为继续阅读入口，框架 API 以你安装版本的文档为准。本手册的项目描述依据本地代码核对；没有声称实时核验所有外部文档或真实模型服务。